

## Exam: Time Series Visualization with Bokeh

This exam tests your ability to visualize time series data using the Bokeh library. You will be working with the "Daily Minimum Temperatures in Melbourne" dataset.

Dataset: "daily-minimum-temperatures-in-melbourne.csv"

```
In [17]: # Imports basiques pour les données et graphiques
import pandas as pd
from bokeh.plotting import figure, show
from bokeh.io import output_notebook
from bokeh.models import (
    ColumnDataSource,
    HoverTool,
    DatetimeTickFormatter,
    NumeralTickFormatter,
)
from bokeh.layouts import row, column
from bokeh.transform import factor_cmap

# Configuration de Bokeh pour affichage dans Jupyter
output_notebook()
```



BokehJS 3.8.2 successfully loaded.

```
In [18]: # Chargement des données météorologiques
meteo_df = pd.read_csv("../datasets/daily-minimum-temperatures-in-melbourne.csv")

# Changement du nom des colonnes
meteo_df.columns = ['Date', 'Temperature']

# Conversion de la date
meteo_df['Date'] = pd.to_datetime(meteo_df['Date'])

# Nettoyage des valeurs aberrantes et conversion en nombre
meteo_df['Temperature'] = meteo_df['Temperature'].astype(str).str.replace('?', '', regex=False)
meteo_df['Temperature'] = pd.to_numeric(meteo_df['Temperature'])

print(meteo_df.head())
print(f"\nTaille: {meteo_df.shape}")
meteo_df
```

```
   Date  Temperature
0 1981-01-01        20.7
1 1981-01-02        17.9
2 1981-01-03        18.8
3 1981-01-04        14.6
4 1981-01-05        15.8
```

Taille: (3650, 2)

```
Out[18]:
```

|      | Date       | Temperature |
|------|------------|-------------|
| 0    | 1981-01-01 | 20.7        |
| 1    | 1981-01-02 | 17.9        |
| 2    | 1981-01-03 | 18.8        |
| 3    | 1981-01-04 | 14.6        |
| 4    | 1981-01-05 | 15.8        |
| ...  | ...        | ...         |
| 3645 | 1990-12-27 | 14.0        |
| 3646 | 1990-12-28 | 13.6        |
| 3647 | 1990-12-29 | 13.5        |
| 3648 | 1990-12-30 | 15.7        |
| 3649 | 1990-12-31 | 13.0        |

3650 rows x 2 columns

### Question 1: Basic Time Series Line Plot

Create a basic line plot showing the daily minimum temperature over time.

- Use the 'Date' column on the x-axis and the 'Temperature' column on the y-axis.

- Set the plot title to "Daily Minimum Temperatures".
- Label the x-axis as "Date" and the y-axis as "Temperature (°C)".
- Add tooltips to display the date and temperature when hovering over the line.
- Enable pan, wheel zoom, and reset tools.

```
In [19]: # Préparation de la source de données pour Bokeh
src_q1 = ColumnDataSource(meteo_df)

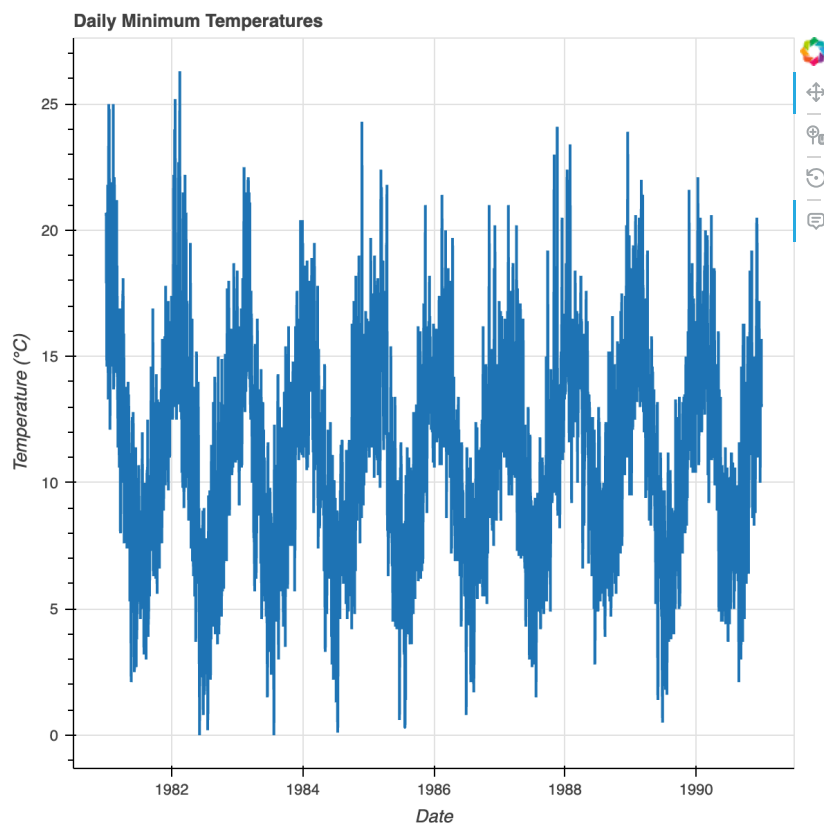
# Création de la figure principale
fig_q1 = figure(
    title="Daily Minimum Temperatures",
    x_axis_type="datetime",
    tools="pan,wheel_zoom,reset"
)

# Tracé de la courbe de température
ligne_q1 = fig_q1.line(
    x="Date",
    y="Temperature",
    source=src_q1,
    line_width=2
)

# Configuration de l'outil de survol
outil_hover1 = HoverTool(
    renderers=[ligne_q1],
    tooltips=[("Date", "@Date{%F}"), ("Temperature", "@Temperature{0.0} °C")],
    formatters={"@Date": "datetime"},
    mode="vline"
)

fig_q1.add_tools(outil_hover1)
fig_q1.xaxis.axis_label = "Date"
fig_q1.yaxis.axis_label = "Temperature (°C)"

show(fig_q1)
```



## Question 2: Rolling Average

Calculate the 30-day rolling average of the daily minimum temperature and plot it alongside the original temperature data.

- Create a new column 'Rolling\_Avg' in the DataFrame containing the 30-day rolling average.
- Plot both the original 'Temperature' and the 'Rolling\_Avg' on the same plot.
- Use different colors and line styles to distinguish between the two.
- Add a legend to the plot to label the lines.
- Add tooltips to display the date, original temperature, and rolling average.

```

In [20]: # Calcul de la moyenne mobile sur 30 jours
meteo_df["Moyenne_Mobile"] = meteo_df["Temperature"].rolling(window=30, min_periods=1).mean()

src_q2 = ColumnDataSource(meteo_df)

fig_q2 = figure(
    title="Daily Temperature and 30-Day Rolling Average",
    x_axis_type="datetime",
    tools="pan,wheel_zoom,reset",
    width=900,
    height=350
)

# Ligne des températures d'origine
ligne_temp = fig_q2.line(
    x="Date",
    y="Temperature",
    source=src_q2,
    line_width=1.8,
    color="blue",
    legend_label="Temperature"
)

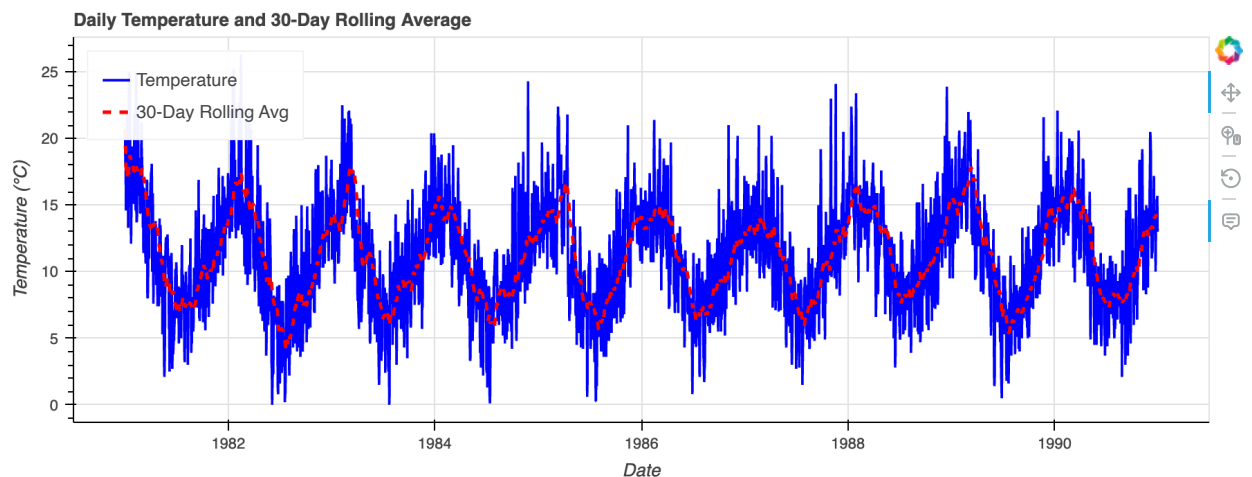
# Ligne de la moyenne mobile
ligne_moyenne = fig_q2.line(
    x="Date",
    y="Moyenne_Mobile",
    source=src_q2,
    line_width=2.4,
    color="red",
    line_dash="dashed",
    legend_label="30-Day Rolling Avg"
)

outil_hover2 = HoverTool(
    renderers=[ligne_temp, ligne_moyenne],
    tooltips=[
        ("Date", "@Date{%F}"),
        ("Temperature", "@Temperature{0.0} °C"),
        ("Rolling Avg", "@Moyenne_Mobile{0.0} °C")
    ],
    formatters={"@Date": "datetime"},
    mode="vline"
)

fig_q2.add_tools(outil_hover2)
fig_q2.legend.location = "top_left"
fig_q2.xaxis.axis_label = "Date"
fig_q2.yaxis.axis_label = "Temperature (°C)"

show(fig_q2)

```



### Question 3: Monthly Box Plots

Create box plots to visualize the distribution of temperatures for each month.

- Extract the month from the 'Date' column and create a new 'Month' column.
- Group the data by 'Month' and prepare it for plotting.
- Use Bokeh's box plot elements to visualize the distribution.
- Label the x-axis with month names and the y-axis with "Temperature (°C)".
- Add tooltips to display the month and relevant statistical values (min, max, median).

```
In [21]: # Extraction du mois
meteo_df["Mois"] = meteo_df["Date"].dt.month_name().str.slice(0, 3)
ordre_mois = ["Jan", "Feb", "Mar", "Apr", "May", "Jun", "Jul", "Aug", "Sep", "Oct", "Nov", "Dec"]

# Statistiques mensuelles pour les boxplots
groupe_mois = meteo_df.groupby("Mois")["Temperature"]
stats_mensuelles = groupe_mois.describe().reset_index()
stats_mensuelles["q1"] = groupe_mois.quantile(0.25).values
stats_mensuelles["q2"] = groupe_mois.quantile(0.50).values
stats_mensuelles["q3"] = groupe_mois.quantile(0.75).values
stats_mensuelles["sup"] = (stats_mensuelles["q3"] + 1.5 * (stats_mensuelles["q3"] - stats_mensuelles["q1"])).clip
stats_mensuelles["inf"] = (stats_mensuelles["q1"] - 1.5 * (stats_mensuelles["q3"] - stats_mensuelles["q1"])).clip
stats_mensuelles["Mois"] = pd.Categorical(stats_mensuelles["Mois"], categories=ordre_mois, ordered=True)
stats_mensuelles = stats_mensuelles.sort_values("Mois")
stats_mensuelles["Mois"] = stats_mensuelles["Mois"].astype(str)

src_q3 = ColumnDataSource(stats_mensuelles)

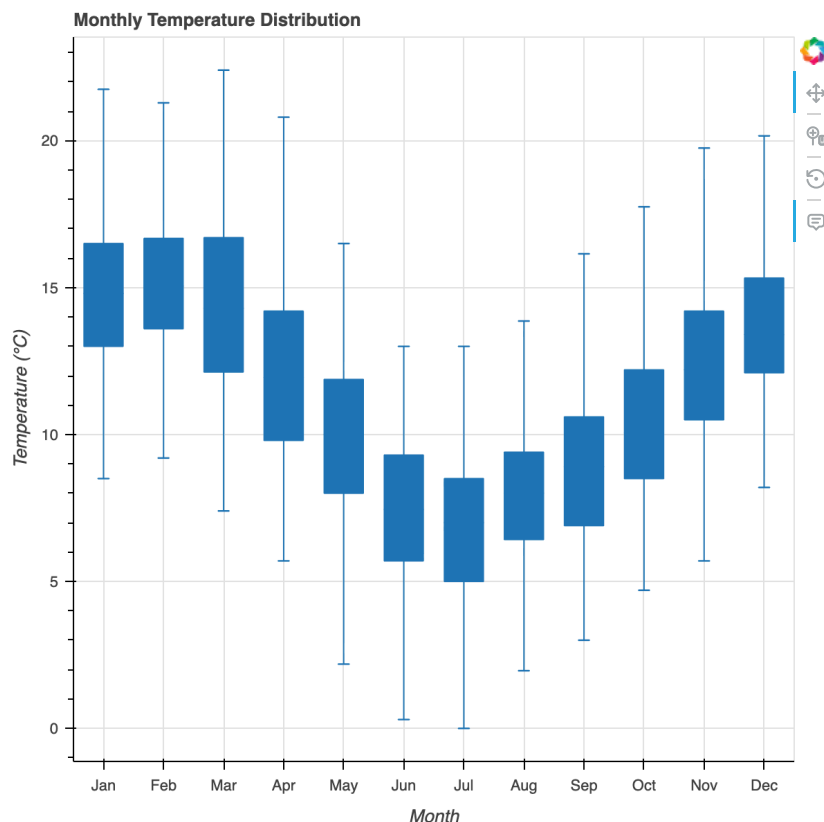
fig_q3 = figure(
    title="Monthly Temperature Distribution",
    x_range=ordre_mois,
    tools="pan,wheel_zoom,reset"
)

# Dessin des boîtes (moustaches et corps)
fig_q3.segment(x0="Mois", y0="sup", x1="Mois", y1="q3", source=src_q3)
fig_q3.segment(x0="Mois", y0="inf", x1="Mois", y1="q1", source=src_q3)
fig_q3.vbar(x="Mois", width=0.65, top="q3", bottom="q2", source=src_q3)
fig_q3.vbar(x="Mois", width=0.65, top="q2", bottom="q1", source=src_q3)
fig_q3.rect(x="Mois", y="inf", width=0.18, height=0.01, source=src_q3)
fig_q3.rect(x="Mois", y="sup", width=0.18, height=0.01, source=src_q3)

outil_hover3 = HoverTool(
    tooltips=[
        ("Month", "@Mois"),
        ("Min", "@min{0.0} °C"),
        ("Max", "@max{0.0} °C"),
        ("Median", "@q2{0.0} °C")
    ]
)

fig_q3.add_tools(outil_hover3)
fig_q3.xaxis.axis_label = "Month"
fig_q3.yaxis.axis_label = "Temperature (°C)"

show(fig_q3)
```



#### Question 4: Yearly Box Plots with Color Mapping

Create box plots to visualize the distribution of temperatures for each year, and use color mapping to highlight temperature variations.

- Extract the year from the 'Date' column and create a new 'Year' column.
- Group the data by 'Year' and prepare it for plotting.
- Use Bokeh's box plot elements to visualize the distribution for each year.
- Label the x-axis with the 'Year' and the y-axis with "Temperature (°C)".
- Use factor\_cmap to color the boxes based on the median temperature of each year.
- Add tooltips to display the year and relevant statistical values (min, max, median, etc.).
- Enable pan, wheel zoom, and reset tools.

```
In [22]: # Extraction de l'année
meteo_df["Annee"] = meteo_df["Date"].dt.year.astype(str)

groupe_annee = meteo_df.groupby("Annee")["Temperature"]
stats_annuelles = groupe_annee.describe().reset_index()
stats_annuelles["q1"] = groupe_annee.quantile(0.25).values
stats_annuelles["q2"] = groupe_annee.quantile(0.50).values
stats_annuelles["q3"] = groupe_annee.quantile(0.75).values
stats_annuelles["sup"] = (stats_annuelles["q3"] + 1.5 * (stats_annuelles["q3"] - stats_annuelles["q1"])).clip(upper=stats_annuelles["q3"] + 1.5 * (stats_annuelles["q3"] - stats_annuelles["q1"]))
stats_annuelles["inf"] = (stats_annuelles["q1"] - 1.5 * (stats_annuelles["q3"] - stats_annuelles["q1"])).clip(lower=stats_annuelles["q1"] - 1.5 * (stats_annuelles["q3"] - stats_annuelles["q1"]))

q_bas = stats_annuelles["q2"].quantile(0.33)
q_haut = stats_annuelles["q2"].quantile(0.66)
stats_annuelles["niveau_median"] = pd.cut(
    stats_annuelles["q2"],
    bins=[-1e9, q_bas, q_haut, 1e9],
    labels=["Low", "Medium", "High"],
    include_lowest=True
).astype(str)

ordre_annee = stats_annuelles["Annee"].tolist()
src_q4 = ColumnDataSource(stats_annuelles)
palette_couleurs = factor_cmap("niveau_median", palette=["blue", "yellow", "red"], factors=["Low", "Medium", "High"])

fig_q4 = figure(
    title="Yearly Temperature Distribution",
    x_range=ordre_annee,
    tools="pan,wheel_zoom,reset",
    width=950,
    height=380
)

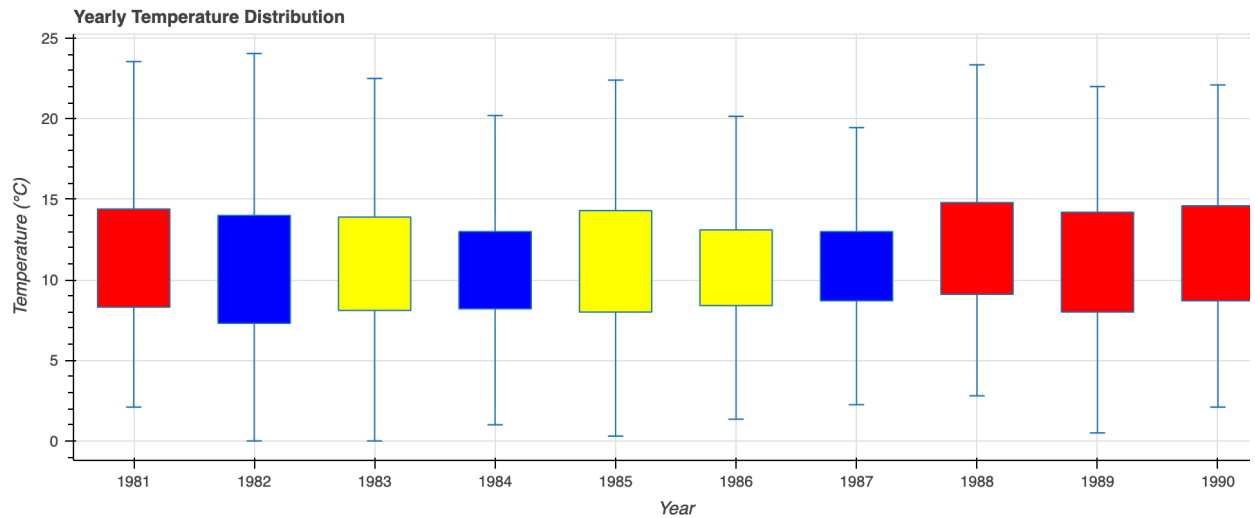
# Boîtes à moustaches avec couleurs thématiques
fig_q4.segment(x0="Annee", y0="sup", x1="Annee", y1="q3", source=src_q4)
fig_q4.segment(x0="Annee", y0="inf", x1="Annee", y1="q1", source=src_q4)
barres_q4 = fig_q4.vbar(
    x="Annee",
    width=0.6,
    top="q3",
    bottom="q1",
    source=src_q4,
    fill_color=palette_couleurs
)

fig_q4.rect(x="Annee", y="inf", width=0.12, height=0.01, source=src_q4)
fig_q4.rect(x="Annee", y="sup", width=0.12, height=0.01, source=src_q4)

outil_hover4 = HoverTool(
    renderers=[barres_q4],
    tooltips=[
        ("Year", "@Annee"),
        ("Min", "@min{0.0} °C"),
        ("Max", "@max{0.0} °C"),
        ("Median", "@q2{0.0} °C"),
        ("Q1", "@q1{0.0} °C"),
        ("Q3", "@q3{0.0} °C"),
        ("Level", "@niveau_median")
    ]
)

fig_q4.add_tools(outil_hover4)
fig_q4.xaxis.axis_label = "Year"
fig_q4.yaxis.axis_label = "Temperature (°C)"

show(fig_q4)
```



## Question 5: Interactive Time Range Selection

Create an interactive line plot where the user can select a specific time range to view using a date range slider.

- Create a basic line plot of 'Temperature' over 'Date'.
- Implement a date range slider using Bokeh widgets to allow users to select a start and end date.
- Update the plot dynamically based on the selected date range.
- Add tooltips to display the date and temperature.
- Enable pan, wheel zoom, and reset tools.

```
In [23]: from bokeh.models import DateRangeSlider

# Données pour l'exploration interactive
src_q5 = ColumnDataSource(meteo_df)

fig_q5 = figure(
    title="Interactive Daily Minimum Temperatures",
    x_axis_type="datetime",
    tools="pan,wheel_zoom,reset"
)

fig_q5.line(
    x="Date",
    y="Temperature",
    source=src_q5,
    line_width=2
)

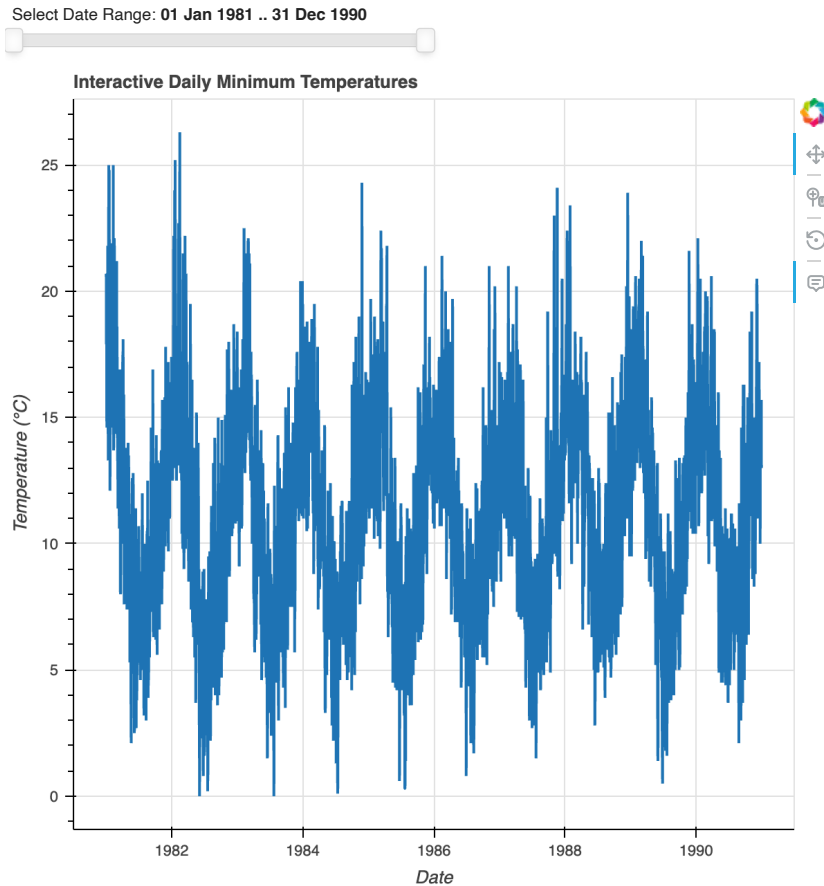
outil_hover5 = HoverTool(
    tooltips=[("Date", "@Date{%F}"), ("Temperature", "@Temperature{0.0} °C")],
    formatters={"@Date": "datetime"},
    mode="vline"
)

fig_q5.add_tools(outil_hover5)
fig_q5.xaxis.axis_label = "Date"
fig_q5.yaxis.axis_label = "Temperature (°C)"

# Slider permettant de sélectionner la plage de dates
selecteur_dates = DateRangeSlider(
    title="Select Date Range",
    start=meteo_df["Date"].min(),
    end=meteo_df["Date"].max(),
    value=(meteo_df["Date"].min(), meteo_df["Date"].max())
)

# Liaison du slider aux axes x du graphique
selecteur_dates.js_link("value", fig_q5.x_range, "start", attr_selector=0)
selecteur_dates.js_link("value", fig_q5.x_range, "end", attr_selector=1)

show(column(selecteur_dates, fig_q5))
```



## Question 6: Time Series Decomposition Visualization

Perform a simple time series decomposition to visualize the trend and seasonality components of the temperature data.

- Resample the data to monthly frequency and calculate the monthly average temperature.
- Use a simple moving average to estimate the trend component.
- Calculate the seasonal component by subtracting the trend from the original monthly data.
- Create three separate Bokeh plots: one for the original monthly data, one for the trend, and one for the seasonal component.
- Ensure the plots are aligned and share the same x-axis (Date).
- Add tooltips to each plot to display the date and corresponding value.
- Enable pan, wheel zoom, and reset tools for each plot.

```
In [24]: # Calcul des valeurs mensuelles pour la décomposition TS
donnees_mensuelles = meteo_df.set_index("Date")["Temperature"].resample("ME").mean().reset_index()
donnees_mensuelles["Tendance"] = donnees_mensuelles["Temperature"].rolling(window=12, min_periods=1, center=True)
donnees_mensuelles["Saisonnier"] = donnees_mensuelles["Temperature"] - donnees_mensuelles["Tendance"]

src_q6 = ColumnDataSource(donnees_mensuelles)

# Graphique de la moyenne mensuelle
fig_moyenne = figure(
    title="Monthly Average Temperature",
    x_axis_type="datetime",
    tools="pan,wheel_zoom,reset"
)
fig_moyenne.line("Date", "Temperature", source=src_q6, line_width=2)
hover_moyenne = HoverTool(
    tooltips=[("Date", "@Date{%F}"), ("Monthly Temp", "@Temperature{0.0} °C")],
    formatters={"@Date": "datetime"},
    mode="vline"
)
fig_moyenne.add_tools(hover_moyenne)
fig_moyenne.xaxis.axis_label = "Date"
fig_moyenne.yaxis.axis_label = "Temperature (°C)"

# Graphique de la tendance
fig_tendance = figure(
    title="Trend Component",
    x_axis_type="datetime",
    tools="pan,wheel_zoom,reset",
    x_range=fig_moyenne.x_range
)
fig_tendance.line("Date", "Tendance", source=src_q6, line_width=2)
hover_tend = HoverTool(
```

```
tooltips=[("Date", "@Date{%F}"), ("Trend", "@Tendance{0.0} °C")],
formatters={"@Date": "datetime"},
mode="vline"
)
fig_tendance.add_tools(hover_tend)
fig_tendance.xaxis.axis_label = "Date"
fig_tendance.yaxis.axis_label = "Trend"

# Graphique de la composante saisonnière
fig_saison = figure(
    title="Seasonal Component",
    x_axis_type="datetime",
    tools="pan,wheel_zoom,reset",
    x_range=fig_moyenne.x_range
)
fig_saison.line("Date", "Saisonnier", source=src_q6, line_width=2)
hover_saison = HoverTool(
    tooltips=[("Date", "@Date{%F}"), ("Seasonal", "@Saisonnier{0.0} °C")],
    formatters={"@Date": "datetime"},
    mode="vline"
)
fig_saison.add_tools(hover_saison)
fig_saison.xaxis.axis_label = "Date"
fig_saison.yaxis.axis_label = "Seasonal"

# Affichage des 3 graphiques alignés
show(column(fig_moyenne, fig_tendance, fig_saison))
```



